RAJALAKSHMI ENGINEERING COLLEGE

RAJALAKSHMI NAGAR, THANDALAM – 602 105



CS23331 - DESIGN AND ANALYSIS OF ALGORITHMS

Laboratory Record Note Book

Name : Kamesh S

Year / Branch / Section : II / CSE / F

University Register No. : 2116250701312

College Roll No. : 250701312

Semester : 3rd

Academic Year : 2025-2029

INDEX PAGE

Exp.No	Name of the Experiment	Date of Completion
1	Finding Complexity using Counter Method	5/8/26
2	Finding Complexity using Counter method	10/8/26
3	Finding Complexity using Counter method	10/8/26
4	Finding Complexity using Counter method	10/8/26
5	Finding Complexity using Counter method	10/8/26
6	1-G-Coin Problem	30/8/26
7	2-G-Cookies Problem	30/8/26
8	3-G-Burger Problem	30/8/26
9	4-G-Array Sum max problem	30/8/26
10	5-G-Product of Array elementsMinimum	30/8/26
11	1-Number of Zeros in a Given Array	4/9/26
12	2-Majority Element	4/9/26
13	3-Finding Floor Value	4/9/26
14	4-Two Elements sum to x	4/9/26
15	5-Implementation of Quick Sort	4/9/26
16	1-DP-Playing with Numbers	29/9/26
17	2-DP-Playing with chessboard	29/9/26
18	3-DP-Longest Common Subsequence	29/9/26
19	4-DP-Longest non-decreasing Subsequence	7/10/26
20	1-Finding Duplicates-O(n^2) Time Complexity,O(1) Space Complexity	29/9/26
21	2-Finding Duplicates-O(n) Time Complexity,O(1) Space Complexity	7/10/26
22	3-Print Intersection of 2 sorted arrays-O(m*n)Time Complexity,O(1) Space Complexity	7/10/26
23	4-Print Intersection of 2 sorted arrays-O(m+n)Time Complexity,O(1) Space Complexity	7/10/26
24	5-Pair with DifferenceO(n^2)Time Complexity,O(1) Space Complexity	7/10/26
25	6-Pair with Difference -O(n) Time Complexity,O(1) Space Complexity	7/10/26

DESIGN ANALYSIS AND ALGORITHM

TIME COMPLEXITY

problem 1

Question 1 | Correct Mark 1.00 out of 1.00 Flag question

Convert the following algorithm into a program and find its time complexity using the counter method.

```
void function (int n)
{
    int i= 1;

    int s =1;

    while(s <= n)
    {
         i++;
         s += i;
     }
}
```

Note: No need of counter increment for declarations and scanf() and count variable printf() statements.

Input:

A positive Integer n

Output:

Print the value of the counter variable

For example:

Input	Result
9	12

```c
#include <stdio.h>

int main() {
    int n;
    int count = 0;


    scanf("%d", &n);

    int i = 1;
    count++;

    int s = 1;
    count++;

    while (s <= n) {
        count++;

        i++;
        count++;

        s += i;
        count++;
    }
    count++;


    printf("%d\n", count);

    return 0;
}
```

	Input	Expected	Got	
✔	9	12	12	✔
✔	4	9	9	✔

Passed all tests! ✔

problem 2

Question 1 | Correct Mark 1.00 out of 1.00 Flag question

Convert the following algorithm into a program and find its time complexity using the counter method.

```
void func(int n)
{
    if(n==1)
    {
      printf("*");
    }
    else
    {
     for(int i=1; i<=n; i++)
     {
       for(int j=1; j<=n; j++)
       {
          printf("*");
          printf("*");
          break;
       }
     }
   }
 }
```

Note: No need of counter increment for declarations and scanf() and count variable printf() statements.

Input:

A positive Integer n

Output:

Print the value of the counter variable

```c
#include <stdio.h>

int main() {
    int n, count = 0;
    scanf("%d", &n);

    count++;
    if(n == 1) {
        count++;
    } else {
        for(int i = 1; i <= n; i++) {
            count++;
            for(int j = 1; j <= n; j++) {
                count++;
                count++;
                count++;
                count++;
                break;
            }
        }
        count++;
    }

    printf("%d\n", count);
    return 0;
}
```

	Input	Expected	Got	
✔	2	12	12	✔
✔	1000	5002	5002	✔
✔	143	717	717	✔

Passed all tests! ✔

Problem 3

Convert the following algorithm into a program and find its time complexity using counter method.

```
Factor(num) {
{
   for (i = 1; i <= num;++i)
   {
    if (num % i== 0)
       {
         printf("%d ", i);
       }
    }
 }
```

Note: No need of counter increment for declarations and scanf() and counter variable printf() statement.

Input:

A positive Integer n

Output:

Print the value of the counter variable

```c
#include <stdio.h>

int main() {
    int n, count = 0;
    scanf("%d", &n);

    for (int i = 1; i <= n; ++i) {
        count++;
        count++;
        if (n % i == 0) {
            count++;
        }
    }
    count++;

    printf("%d\n", count);
    return 0;
}
```

	Input	Expected	Got	
✔	12	31	31	✔
✔	25	54	54	✔
✔	4	12	12	✔

Passed all tests! ✔

Problem 4

Convert the following algorithm into a program and find its time

complexity using counter method.

```
void function(int n)
{
    int c= 0;
    for(int i=n/2; i<n; i++)
        for(int j=1; j<n; j = 2 * j)
            for(int k=1; k<n; k = k * 2)
                c++;
}
```

Note: No need of counter increment for declarations and scanf() and count variable printf() statements.

Input:

A positive Integer n

Output:

Print the value of the counter variable

```c
#include <stdio.h>

int main() {
    int n, count = 0;
    scanf("%d", &n);

    int c = 0;
    count++;

    for (int i = n / 2; i < n; i++) {
        count++;
        for (int j = 1; j < n; j = 2 * j) {
            count++;
            for (int k = 1; k < n; k = k * 2) {
                count++;
                c++;
                count++;
            }
            count++;
        }
        count++;
    }
    count++;

    printf("%d\n", count);
    return 0;
}
```

	Input	Expected	Got	
✔	4	30	30	✔
✔	10	212	212	✔

Passed all tests! ✔

Problem 5

Convert the following algorithm into a program and find its time complexity using counter method.

```
void reverse(int n)
{
   int rev = 0, remainder;
   while (n != 0)
    {
        remainder = n % 10;
        rev = rev * 10 + remainder;
        n/= 10;

    }
print(rev);
}
```

Note: No need of counter increment for declarations and scanf() and count variable printf() statements.

Input:

A positive Integer n

Output:

Print the value of the counter variable

```c
#include <stdio.h>

int main() {
    int n, count = 0;
    scanf("%d", &n);

    int rev = 0, remainder;
    count++;

    while (n != 0) {
        count++;
        remainder = n % 10;
        count++;
        rev = rev * 10 + remainder;
        count++;
        n /= 10;
        count++;
    }
    count++;
    count++;

    printf("%d\n", count);
    return 0;
}
```

	Input	Expected	Got	
✔	12	11	11	✔
✔	1234	19	19	✔

Passed all tests! ✔

DESIGN ANALYSIS AND ALGORITHM

GREEDY ALGORITHMS

Problem 1 :

Write a program to take value V and we want to make change for V Rs, and we have infinite supply of each of the denominations in Indian currency, i.e., we have infinite supply of { 1, 2, 5, 10, 20, 50, 100, 500, 1000} valued coins/notes, what is the minimum number of coins and/or notes needed to make the change.

Input Format:

Take an integer from stdin.

Output Format:

print the integer which is change of the number.

Example Input :

64

Output:

4

Explanaton:

We need a 50 Rs note and a 10 Rs note and two 2 rupee coins.

```c
#include <stdio.h>

int main() {
    int v;
    scanf("%d", &v);

    int denominations[] = {1000, 500, 100, 50, 20, 10, 5, 2, 1};
    int count = 0;

    for (int i = 0; i < 9; i++) {
        if (v >= denominations[i]) {
            count += v / denominations[i];
            v = v % denominations[i];
        }
    }

    printf("%d\n", count);
    return 0;
}
```

	Input	Expected	Got	
✔	49	5	5	✔

Passed all tests! ✔

Problem 2:

Assume you are an awesome parent and want to give your children some cookies. But, you should give each child at most one cookie.

Each child i has a greed factor g[i], which is the minimum size of a cookie that the child will be content with; and each cookie j has a size s[j]. If s[j] >= g[i], we can assign the cookie j to the child i, and the child i will be content. Your goal is to maximize the number of your content children and output the maximum number.

Example 1:

Input:

3

1 2 3

2

1 1

Output:

1

Explanation: You have 3 children and 2 cookies. The greed factors of 3 children are 1, 2, 3.

And even though you have 2 cookies, since their size is both 1, you could only make the child whose greed factor is 1 content.

You need to output 1.

Constraints:

1 <= g.length <= 3 * 10^4

0 <= s.length <= 3 * 10^4

1 <= g[i], s[j] <= 2^31 - 1

```c
#include <stdlib.h>

int compare(const void *a, const void *b) {
    int val1 = *(const int *)a;
    int val2 = *(const int *)b;
    if (val1 < val2) return -1;
    if (val1 > val2) return 1;
    return 0;
}

int main() {
    int n_g;
    if (scanf("%d", &n_g) != 1) return 0;

    int *g = (int *)malloc(n_g * sizeof(int));
    for (int i = 0; i < n_g; i++) {
        scanf("%d", &g[i]);
    }

    int n_s;
    if (scanf("%d", &n_s) != 1) return 0;

    int *s = NULL;
    if (n_s > 0) {
        s = (int *)malloc(n_s * sizeof(int));
        for (int i = 0; i < n_s; i++) {
            scanf("%d", &s[i]);
        }
    }

    qsort(g, n_g, sizeof(int), compare);
    if (n_s > 0) {
        qsort(s, n_s, sizeof(int), compare);
    }

    int i = 0, j = 0;
    while (i < n_g && j < n_s) {
        if (s[j] >= g[i]) {
            i++;
        }
        j++;
    }
```

```
44
45      printf("%d\n", i);
46
47      free(g);
48      if (s != NULL) {
49          free(s);
50      }
51
52      return 0;
53  }
```

	Input	Expected	Got	
✔	2 1 2 3 1 2 3	2	2	✔

Passed all tests! ✔

Problem 3:

A person needs to eat burgers. Each burger contains a count of calorie. After eating the burger, the person needs to run a distance to burn out his calories. If he has eaten i burgers with c calories each, then he has to run at least $3^i * c$ kilometers to burn out the calories. For example, if he ate 3 burgers with the count of calorie in the order: [1, 3, 2], the kilometers he needs to run are $(3^0 * 1) + (3^1 * 3) + (3^2 * 2) = 1 + 9 + 18 = 28$. But this is not the minimum, so need to try out other orders of consumption and choose the minimum value. Determine the minimum distance he needs to run. Note: He can eat burger in any order and use an efficient sorting algorithm.Apply greedy approach to solve the problem.

Input Format

First Line contains the number of burgers

Second line contains calories of each burger which is n space-separate integers

Output Format

Print: Minimum number of kilometers needed to run to burn out the calories

Sample Input

```
3
5 10 7
```

Sample Output

```
76
```

For example:

Test	Input	Result
Test Case 1	3 1 3 2	18

```c
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

int cmp(const void *a, const void *b){
    return(*(int*)b-*(int*)a);
}

int main(){
    int n;
    scanf("%d", &n);
    int calories[n];
    for(int i=0;i<n;i++){
        scanf("%d", &calories[i]);
    }
    qsort(calories, n, sizeof(int), cmp);
    long long min_distance=0;
    for(int i=0; i<n; i++){
        min_distance+=(long long)calories[i] * pow(n,i);
    }
    printf("%lld\n", min_distance);
    return 0;
}
```

	Test	Input	Expected	Got	
✔	Test Case 1	3 1 3 2	18	18	✔
✔	Test Case 2	4 7 4 9 6	389	389	✔
✔	Test Case 3	3 5 10 7	76	76	✔

Passed all tests! ✔

Problem 4:

Given an array of N integer, we have to maximize the sum of arr[i] * i, where i is the index of the element (i = 0, 1, 2, ..., N).Write an algorithm based on Greedy technique with a Complexity O(nlogn).

Input Format:

First line specifies the number of elements-n

The next n lines contain the array elements.

Output Format:

Maximum Array Sum to be printed.

Sample Input:

5

2 5 3 4 0

Sample output:

40

```c
#include <stdio.h>
#include <stdlib.h>

int compare(const void *a, const void *b) {
    long long val1 = *(const long long *)a;
    long long val2 = *(const long long *)b;
    if (val1 < val2) return -1;
    if (val1 > val2) return 1;
    return 0;
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) {
        return 0;
    }

    long long *arr = (long long *)malloc(n * sizeof(long long));
    if (arr == NULL) {
        return 0;
    }

    for (int i = 0; i < n; i++) {
        scanf("%lld", &arr[i]);
    }

    qsort(arr, n, sizeof(long long), compare);

    long long sum = 0;
    for (long long i = 0; i < n; i++) {
        sum += arr[i] * i;
    }

    printf("%lld\n", sum);

    free(arr);
    return 0;
}
```

	Input	Expected	Got	
✔	5 2 5 3 4 0	40	40	✔
✔	10 2 2 2 4 4 3 3 5 5 5	191	191	✔
✔	2 45 3	45	45	✔

Passed all tests! ✔

Problem 5:

Given two arrays array_One[] and array_Two[] of same size N. We need to first rearrange the arrays such that the sum of the product of pairs(1 element from each) is minimum. That is SUM (A[i] * B[i]) for all i is minimum.

For example:

Input	Result
3 1 2 3 4 5 6	28

```c
#include <stdio.h>
#include <stdlib.h>

int compare(const void *a, const void *b) {
    long long val1 = *(const long long *)a;
    long long val2 = *(const long long *)b;
    if (val1 < val2) return -1;
    if (val1 > val2) return 1;
    return 0;
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) {
        return 0;
    }

    long long *A = (long long *)malloc(n * sizeof(long long));
    long long *B = (long long *)malloc(n * sizeof(long long));

    if (A == NULL || B == NULL) {
        return 0;
    }

    for (int i = 0; i < n; i++) {
        scanf("%lld", &A[i]);
    }

    for (int i = 0; i < n; i++) {
        scanf("%lld", &B[i]);
    }

    qsort(A, n, sizeof(long long), compare);
    qsort(B, n, sizeof(long long), compare);

    long long sum = 0;
    for (int i = 0; i < n; i++) {
        sum += A[i] * B[n - 1 - i];
    }

    printf("%lld\n", sum);

    free(A);
    free(B);
    return 0;
}
```

	Input	Expected	Got	
✔	3 1 2 3 4 5 6	28	28	✔
✔	4 7 5 1 2 1 3 4 1	22	22	✔
✔	5 20 10 30 10 40 8 9 4 3 10	590	590	✔

Passed all tests! ✔

DESIGN ANALYSIS AND ALGORITHM

DIVIDE AND CONQUER

Problem 1:

Problem Statement

Given an array of 1s and 0s this has all 1s first followed by all 0s. Aim is to find the number of 0s. Write a program using Divide and Conquer to Count the number of zeroes in the given array.

Input Format

First Line Contains Integer m – Size of array

Next m lines Contains m numbers – Elements of an array

Output Format

First Line Contains Integer – Number of zeroes present in the given array.

```c
#include <stdio.h>

int countZeroes(int arr[], int low, int high, int m) {
    if (high >= low) {
        int mid = low + (high - low) / 2;


        if ((mid == 0 || arr[mid - 1] == 1) && arr[mid] == 0) {
            return m - mid; // Number of zeroes
        }


        if (arr[mid] == 1) {
            return countZeroes(arr, mid + 1, high, m);
        }

        else {
            return countZeroes(arr, low, mid - 1, m);
        }
    }


    return 0;
}

int main() {
    int m;


    if (scanf("%d", &m) != 1) {
        return 1;
    }

    int arr[m];


    for (int i = 0; i < m; i++) {
        scanf("%d", &arr[i]);
    }


    int zeroesCount = countZeroes(arr, 0, m - 1, m);
    printf("%d\n", zeroesCount);

    return 0;
}
```

	Input	Expected	Got	
✔	5 1 1 1 0 0	2	2	✔
✔	10 1 1 1 1 1 1 1 1 1 1	0	0	✔
✔	8 0 0 0 0 0 0 0 0 0	8	8	✔

Problem 2:

Given an array `nums` of size `n`, return *the majority element*.

The majority element is the element that appears more than `⌊n / 2⌋` times. You may assume that the majority element always exists in the array.

Example 1:

```
Input: nums = [3,2,3]
Output: 3
```

Example 2:

```
Input: nums = [2,2,1,1,1,2,2]
Output: 2
```

Constraints:

- `n == nums.length`
- `1 <= n <= 5 * 10^4`
- `-2^31 <= nums[i] <= 2^31 - 1`

For example:

Input	Result
3 3 2 3	3
7 2 2 1 1 1 2 2	2

```c
#include <stdio.h>

int majorityElement(int* nums, int numsSize) {
    int count = 0;
    int candidate = 0;

    for (int i = 0; i < numsSize; i++) {
        if (count == 0) {
            candidate = nums[i];
        }
        count += (nums[i] == candidate) ? 1 : -1;
    }

    return candidate;
}

int main() {
    int n;

    // Read the number of elements
    if (scanf("%d", &n) != 1) {
        return 1;
    }

    int nums[n];

    // Read the array elements
    for (int i = 0; i < n; i++) {
        scanf("%d", &nums[i]);
    }

    // Print ONLY the majority element
    printf("%d\n", majorityElement(nums, n));

    return 0;
}
```

	Input	Expected	Got	
✔	3 3 2 3	3	3	✔

Passed all tests! ✔

Problem 3:

Problem Statement:

Given a sorted array and a value x, the floor of x is the largest element in array smaller than or equal to x. Write divide and conquer algorithm to find floor of x.

Input Format

First Line Contains Integer n – Size of array

Next n lines Contains n numbers – Elements of an array

Last Line Contains Integer x – Value for x

Output Format

First Line Contains Integer – Floor value for x

```c
#include <stdio.h>

int findFloor(int arr[], int n, int x) {
    int low = 0;
    int high = n - 1;
    int floor_val = -1;

    while (low <= high) {
        int mid = low + (high - low) / 2;

        if (arr[mid] == x) {
            return arr[mid];
        }

        if (arr[mid] < x) {
            floor_val = arr[mid];
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }

    return floor_val;
}

int main() {
    int n, x;

    if (scanf("%d", &n) != 1) {
        return 1;
    }

    int arr[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    if (scanf("%d", &x) != 1) {
        return 1;
    }

    printf("%d\n", findFloor(arr, n, x));

    return 0;
}
```

	Input	Expected	Got	
✔	6 1 2 8 10 12 19 5	2	2	✔
✔	5 10 22 85 108 129 100	85	85	✔

Problem 4:

Problem Statement:

Given a sorted array of integers say arr[] and a number x. Write a recursive program using divide and conquer strategy to check if there exist two elements in the array whose sum = x. If there exist such two elements then return the numbers, otherwise print as "No".

Note: Write a Divide and Conquer Solution

Input Format

First Line Contains Integer n – Size of array

Next n lines Contains n numbers – Elements of an array

Last Line Contains Integer x – Sum Value

Output Format

First Line Contains Integer – Element1

Second Line Contains Integer – Element2 (Element 1 and Elements 2 together sums to value "x")

```c
#include <stdio.h>

// Recursive approach to narrow down the search space (Decrease/Divide and Conquer)
int findPair(int arr[], int left, int right, int x) {
    // Base case: if the pointers cross, no pair exists
    if (left >= right) {
        return 0;
    }

    int sum = arr[left] + arr[right];

    // If the sum matches x, print the elements and return true
    if (sum == x) {
        printf("%d\n%d\n", arr[left], arr[right]);
        return 1;
    }
    // If sum is less than x, search in the remaining rightward space
    else if (sum < x) {
        return findPair(arr, left + 1, right, x);
    }
    // If sum is greater than x, search in the remaining leftward space
    else {
        return findPair(arr, left, right - 1, x);
    }
}

int main() {
    int n, x;

    // Read size of the array
    if (scanf("%d", &n) != 1) {
        return 1;
    }

    int arr[n];

    // Read array elements
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }
```

```
    }

    // Read the target sum value
    if (scanf("%d", &x) != 1) {
        return 1;
    }

    // Call the recursive function; if it returns 0, print "No"
    if (!findPair(arr, 0, n - 1, x)) {
        printf("No\n");
    }

    return 0;
}
```

	Input	Expected	Got	
✔	4 2 4 8 10 14	4 10	4 10	✔
✔	5 2 4 6 8 10 100	No	No	✔

Passed all tests! ✔

Problem 5:

Write a Program to Implement the Quick Sort Algorithm

Input Format:
The first line contains the no of elements in the list-n
The next n lines contain the elements.

Output:
Sorted list of elements

For example:

Input	Result
5 67 34 12 98 78	12 34 67 78 98

```c
#include <stdio.h>


void swap(int* a, int* b) {
    int t = *a;
    *a = *b;
    *b = t;
}


int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = (low - 1);

    for (int j = low; j <= high - 1; j++) {

        if (arr[j] < pivot) {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i + 1], &arr[high]);
    return (i + 1);
}


void quickSort(int arr[], int low, int high) {
    if (low < high) {

        int pi = partition(arr, low, high);


        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

int main() {
    int n;


    if (scanf("%d", &n) != 1) {
        return 1;
    }

    int arr[n];


    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }


    quickSort(arr, 0, n - 1);


    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");

    return 0;
}
```

DESIGN ANALYSIS AND ALGORITHM

DYNAMIC PROGRAMMING

Problem 1:

Playing with Numbers:

Ram and Sita are playing with numbers by giving puzzles to each other. Now it was Ram term, so he gave Sita a positive integer 'n' and two numbers 1 and 3. He asked her to find the possible ways by which the number n can be represented using 1 and 3.Write any efficient algorithm to find the possible ways.

Example 1:

Input: *6*
Output:*6*
Explanation: *There are 6 ways to 6 represent number with 1 and 3*

1+1+1+1+1+1
3+3
1+1+1+3
1+1+3+1
1+3+1+1
3+1+1+1

Input Format

First Line contains the number n

Output Format

Print: The number of possible ways 'n' can be represented using 1 and 3

Sample Input

6

Sample Output

6

```c
#include <stdio.h>

unsigned long long countWays(int n) {
    unsigned long long dp[n + 1];
    dp[0] = 1;

    for (int i = 1; i <= n; i++) {
        dp[i] = dp[i - 1];
        if (i >= 3)
            dp[i] += dp[i - 3];
    }

    return dp[n];
}

int main() {
    int n;
    scanf("%d", &n);
    printf("%llu\n", countWays(n));
    return 0;
}
```

	Input	Expected	Got	
✔	6	6	6	✔
✔	25	8641	8641	✔
✔	100	24382819596721629	24382819596721629	✔

Passed all tests! ✔

Problem 2:

Playing with Chessboard:

Ram is given with an n*n chessboard with each cell with a monetary value. Ram stands at the (0,0), that the position of the top left white rook. He is been given a task to reach the bottom right black rook position (n-1, n-1) constrained that he needs to reach the position by traveling the maximum monetary path under the condition that he can only travel one step right or one step down the board. Help ram to achieve it by providing an efficient DP algorithm.

Example:

Input

3

1 2 4

2 3 4

8 7 1

Output:

19

Explanation:

Totally there will be 6 paths among that the optimal is

Optimal path value:1+2+8+7+1=19

Input Format

First Line contains the integer n

The next n lines contain the n*n chessboard values

Output Format

Print Maximum monetary value of the path

```c
#include <stdio.h>

int main() {
    int n;
    if (scanf("%d", &n) != 1) {
        return 0;
    }

    long long grid[n][n];
    long long dp[n][n];

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%lld", &grid[i][j]);
        }
    }

    dp[0][0] = grid[0][0];

    for (int i = 1; i < n; i++) {
        dp[i][0] = dp[i - 1][0] + grid[i][0];
    }

    for (int j = 1; j < n; j++) {
        dp[0][j] = dp[0][j - 1] + grid[0][j];
    }

    for (int i = 1; i < n; i++) {
        for (int j = 1; j < n; j++) {
            long long from_top = dp[i - 1][j];
            long long from_left = dp[i][j - 1];
            dp[i][j] = (from_top > from_left ? from_top : from_left) + grid[i][j];
        }
    }

    printf("%lld\n", dp[n - 1][n - 1]);

    return 0;
}
```

	Input	Expected	Got	
✔	3 1 2 4 2 3 4 8 7 1	19	19	✔
✔	3 1 3 1 1 5 1 4 2 1	12	12	✔

Problem 3:

Given two strings find the length of the common longest subsequence(need not be contiguous) between the two.

Example:

s1: ggtabe

s2: tgatasb

s1	a	g	**g**	**t**	**a**	**b**	
s2	**g**	x	**t**	x	**a**	y	**b**

The length is 4

Solveing it using Dynamic Programming

For example:

Input	Result
aab azb	2

```c
#include <stdio.h>
#include <string.h>

int max(int a, int b) {
    return (a > b) ? a : b;
}

int main() {
    char s1[10005], s2[10005];
    if (scanf("%s", s1) != 1) return 0;
    if (scanf("%s", s2) != 1) return 0;

    int m = strlen(s1);
    int n = strlen(s2);

    int dp[m + 1][n + 1];

    for (int i = 0; i <= m; i++) {
        for (int j = 0; j <= n; j++) {
            if (i == 0 || j == 0) {
                dp[i][j] = 0;
            } else if (s1[i - 1] == s2[j - 1]) {
                dp[i][j] = dp[i - 1][j - 1] + 1;
            } else {
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
            }
        }
    }

    printf("%d\n", dp[m][n]);

    return 0;
}
```

	Input	Expected	Got	
✔	aab azb	2	2	✔
✔	ABCD ABCD	4	4	✔

Passed all tests! ✔

Problem 4:

Problem statement:

Find the length of the Longest Non-decreasing Subsequence in a given Sequence.

Eg:

Input:9

Sequence:[-1,3,4,5,2,2,2,2,3]

the subsequence is [-1,2,2,2,2,3]

Output:6

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n;
    if (scanf("%d", &n) != 1) {
        return 0;
    }

    if (n <= 0) {
        printf("0\n");
        return 0;
    }

    int *arr = (int *)malloc(n * sizeof(int));
    int *dp = (int *)malloc(n * sizeof(int));

    if (arr == NULL || dp == NULL) {
        return 0;
    }

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
        dp[i] = 1;
    }

    int max_len = 1;

    for (int i = 1; i < n; i++) {
        for (int j = 0; j < i; j++) {
            if (arr[i] >= arr[j] && dp[i] < dp[j] + 1) {
                dp[i] = dp[j] + 1;
            }
        }
        if (dp[i] > max_len) {
            max_len = dp[i];
        }
    }

    printf("%d\n", max_len);

    free(arr);
    free(dp);

    return 0;
}
```

	Input	Expected	Got	
✔	9 -1 3 4 5 2 2 2 2 3	6	6	✔
✔	7 1 2 2 4 5 7 6	6	6	✔

Passed all tests! ✔

DESIGN ANALYSIS AND ALGORITHM

COMPETITVE PROGRAMMING

Problem 1:

Find Duplicate in Array.

Given a read only array of n integers between 1 and n, find one number that repeats.

Input Format:

First Line - Number of elements

n Lines - n Elements

Output Format:

Element x - That is repeated

For example:

Input	Result
5 1 1 2 3 4	1

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n;
    if (scanf("%d", &n) != 1) {
        return 0;
    }

    int *freq = (int *)calloc(n + 1, sizeof(int));
    if (freq == NULL) {
        return 0;
    }

    int duplicate = -1;
    for (int i = 0; i < n; i++) {
        int val;
        scanf("%d", &val);
        if (freq[val] == 1) {
            duplicate = val;
            break;
        }
        freq[val] = 1;
    }

    if (duplicate != -1) {
        printf("%d\n", duplicate);
    }

    free(freq);
    return 0;
}
```

	Input	Expected	Got	
✔	11 10 9 7 6 5 1 2 3 8 4 7	7	7	✔
✔	5 1 2 3 4 4	4	4	✔
✔	5 1 1 2 3 4	1	1	✔

Passed all tests! ✔

Problem 2:

Find Duplicate in Array.

Given a read only array of n integers between 1 and n, find one number that repeats.

Input Format:

First Line - Number of elements

n Lines - n Elements

Output Format:

Element x - That is repeated

For example:

Input	Result
5 1 1 2 3 4	1

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n;
    if (scanf("%d", &n) != 1) {
        return 0;
    }

    int *freq = (int *)calloc(n + 1, sizeof(int));
    if (freq == NULL) {
        return 0;
    }

    int duplicate = -1;
    for (int i = 0; i < n; i++) {
        int val;
        scanf("%d", &val);
        if (freq[val] == 1) {
            duplicate = val;
        }
        freq[val] = 1;
    }

    if (duplicate != -1) {
        printf("%d\n", duplicate);
    }

    free(freq);
    return 0;
}
```

	Input	Expected	Got	
✔	11 10 9 7 6 5 1 2 3 8 4 7	7	7	✔
✔	5 1 2 3 4 4	4	4	✔
✔	5 1 1 2 3 4	1	1	✔

Passed all tests! ✔

Problem 3:

Find the intersection of two sorted arrays.

OR in other words,

Given 2 sorted arrays, find all the elements which occur in both the arrays.

Input Format

- The first line contains T, the number of test cases. Following T lines contain:

1. Line 1 contains N1, followed by N1 integers of the first array
2. Line 2 contains N2, followed by N2 integers of the second array

Output Format

The intersection of the arrays in a single line

Example

Input:

1

3 10 17 57

6 2 7 10 15 57 246

Output:

10 57

Input:

1

6 1 2 3 4 5 6

2 1 6

Output:

1 6

For example:

Input	Result
1 3 10 17 57 6 2 7 10 15 57 246	10 57

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int t;
    if (scanf("%d", &t) != 1) return 0;

    while (t--) {
        int n1;
        scanf("%d", &n1);
        int *arr1 = (int *)malloc(n1 * sizeof(int));
        for (int i = 0; i < n1; i++) {
            scanf("%d", &arr1[i]);
        }

        int n2;
        scanf("%d", &n2);
        int *arr2 = (int *)malloc(n2 * sizeof(int));
        for (int i = 0; i < n2; i++) {
            scanf("%d", &arr2[i]);
        }

        int i = 0, j = 0;
        int printed = 0;

        while (i < n1 && j < n2) {
            if (arr1[i] < arr2[j]) {
                i++;
            } else if (arr1[i] > arr2[j]) {
                j++;
            } else {
                if (printed) {
                    printf(" ");
                }
                printf("%d", arr1[i]);
                printed = 1;
                i++;
                j++;
            }
        }
        printf("\n");

        free(arr1);
        free(arr2);
    }

    return 0;
}
```

	Input	Expected	Got	
✔	1 3 10 17 57 6 2 7 10 15 57 246	10 57	10 57	✔
✔	1 6 1 2 3 4 5 6 2 1 6	1 6	1 6	✔

Passed all tests! ✔

Problem 4:

Find the intersection of two sorted arrays.

OR in other words,

Given 2 sorted arrays, find all the elements which occur in both the arrays.

Input Format

· The first line contains T, the number of test cases. Following T lines contain:

1. Line 1 contains N1, followed by N1 integers of the first array
2. Line 2 contains N2, followed by N2 integers of the second array

Output Format

The intersection of the arrays in a single line

Example

Input:

1

3 10 17 57

6 2 7 10 15 57 246

Output:

10 57

Input:

1

6 1 2 3 4 5 6

2 1 6

Output:

1 6

For example:

Input	Result
1 3 10 17 57 6 2 7 10 15 57 246	10 57

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int t;
    if (scanf("%d", &t) != 1) return 0;

    while (t--) {
        int n1;
        scanf("%d", &n1);
        int *arr1 = (int *)malloc(n1 * sizeof(int));
        for (int i = 0; i < n1; i++) {
            scanf("%d", &arr1[i]);
        }

        int n2;
        scanf("%d", &n2);
        int *arr2 = (int *)malloc(n2 * sizeof(int));
        for (int i = 0; i < n2; i++) {
            scanf("%d", &arr2[i]);
        }

        int i = 0, j = 0;
        int printed = 0;

        while (i < n1 && j < n2) {
            if (arr1[i] < arr2[j]) {
                i++;
            } else if (arr1[i] > arr2[j]) {
                j++;
            } else {
                if (printed) {
                    printf(" ");
                }
                printf("%d", arr1[i]);
                printed = 1;
                i++;
                j++;
            }
        }
        printf("\n");

        free(arr1);
        free(arr2);
    }

    return 0;
}
```

	Input	Expected	Got	
✔	1 3 10 17 57 6 2 7 10 15 57 246	10 57	10 57	✔
✔	1 6 1 2 3 4 5 6 2 1 6	1 6	1 6	✔

Passed all tests! ✔

Problem 5:

Given an array A of sorted integers and another non negative integer k, find if there exists 2 indices i and j such that A[j] - A[i] = k, i != j.

Input Format:

First Line n - Number of elements in an array

Next n Lines - N elements in the array

k - Non - Negative Integer

Output Format:

1 - If pair exists

0 - If no pair exists

Explanation for the given Sample Testcase:

YES as 5 - 1 = 4

So Return 1.

For example:

Input	Result
3 1 3 5 4	1

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n;
    if (scanf("%d", &n) != 1) {
        return 0;
    }

    int *A = (int *)malloc(n * sizeof(int));
    if (A == NULL) {
        return 0;
    }

    for (int x = 0; x < n; x++) {
        scanf("%d", &A[x]);
    }

    int k;
    if (scanf("%d", &k) != 1) {
        free(A);
        return 0;
    }

    int i = 0;
    int j = 1;
    int found = 0;

    while (i < n && j < n) {
        if (i == j) {
            j++;
            continue;
        }

        int diff = A[j] - A[i];

        if (diff == k) {
            found = 1;
            break;
        } else if (diff < k) {
            j++;
        } else {
            i++;
        }
    }

    printf("%d\n", found);

    free(A);
    return 0;
}
```

	Input	Expected	Got	
✔	3 1 3 5 4	1	1	✔
✔	10 1 4 6 8 12 14 15 20 21 25 1	1	1	✔

Problem 6:

Given an array A of sorted integers and another non negative integer k, find if there exists 2 indices i and j such that A[j] - A[i] = k, i != j.

Input Format:

First Line n - Number of elements in an array

Next n Lines - N elements in the array

k - Non - Negative Integer

Output Format:

1 - If pair exists

0 - If no pair exists

Explanation for the given Sample Testcase:

YES as 5 - 1 = 4

So Return 1.

For example:

Input	Result
3 1 3 5 4	1

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n;
    if (scanf("%d", &n) != 1) {
        return 0;
    }

    int *A = (int *)malloc(n * sizeof(int));
    if (A == NULL) {
        return 0;
    }

    for (int x = 0; x < n; x++) {
        scanf("%d", &A[x]);
    }

    int k;
    if (scanf("%d", &k) != 1) {
        free(A);
        return 0;
    }

    int i = 0;
    int j = 1;
    int found = 0;

    while (i < n && j < n) {
        if (i == j) {
            j++;
            continue;
        }

        int diff = A[j] - A[i];

        if (diff == k) {
            found = 1;
            break;
        } else if (diff < k) {
            j++;
        } else {
            i++;
        }
    }

    printf("%d\n", found);

    free(A);
    return 0;
}
```

	Input	Expected	Got	
✔	3 1 3 5 4	1	1	✔
✔	10 1 4 6 8 12 14 15 20 21 25 1	1	1	✔
✔	10 1 2 3 5 11 14 16 24 28 29 0	0	0	✔
✔	10 0 2 3 7 13 14 15 20 24 25 10	1	1	✔

Passed all tests! ✔